EECE 7205 – Assignment #01

Sorting-Based Interview Problems

MyungJin Suh

September 23, 2026

All ten solutions are in `cpp/Assignment01_Suh.cpp`, one menu-driven program with each problem in its own function.

```
$ cd cpp
$ g++ -std=c++17 -O2 -Wall -Wextra -o Assignment01_Suh Assignment01_Suh.cpp
$ ./Assignment01_Suh
```

Each problem below gives the approach, the pseudocode, the complexity, and the program's actual output for the provided example plus additional tests including at least one edge case.

Testing. The solutions are covered by an automated test suite in `test/`, run with `./run_tests.sh`:

```
$ cd test
$ ./run_tests.sh
  RESULT: 30 passed, 0 failed
  ALL TESTS PASSED
```

`test_solutions.cpp` runs three cases per problem — the provided example, one additional case and one edge case — each printing its problem, input and output. A failing case also prints the expected value.

Problem 1: Merge Overlapping Intervals

Question. Given an array of intervals $[start, end]$, sort the intervals by starting value and merge every overlapping interval. Return the resulting non-overlapping intervals.

Approach. Sort the intervals by starting value, then sweep once. Because the starts are non-decreasing, each interval can only overlap the block currently being built, so it is compared with the last merged interval only.

Algorithm 1 Merge Overlapping Intervals

```
1:  function MergeIntervals(intervals)
2:      if intervals is empty then
3:          return empty list
4:      end if
5:      Sort intervals by starting value
6:      merged ← empty list
7:      current ← first interval
8:      for each next interval starting from the second interval do
9:          if next.start ≤ current.end then
10:             current.end ← max(current.end, next.end)
11:         else
12:             Add current to merged
13:             current ← next
14:         end if
15:     end for
16:     Add current to merged
17:     return merged
18: end function
```

Why it is correct. After sorting, $next.start \geq current.start$. If $next.start \leq current.end$ the two overlap and their union is $[current.start,\ \max(current.end, next.end)]$; otherwise every remaining interval starts after $current.end$, so $current$ is final. Empty input returns early; a nested interval is handled by the maximum.

Complexity. Time: $O(n \log n)$ – the sort dominates; best, average and worst coincide.
Space: $O(\log n)$ auxiliary, plus the output.

Output.

```
[Test 1] Provided example
  Input : [[1,3], [2,6], [8,10], [15,18]]
  Output: [[1,6], [8,10], [15,18]]

[Test 2] Unsorted, nested and touching blocks
  Input : [[4,5], [1,4], [2,3], [9,9]]
  Output: [[1,5], [9,9]]

[Test 3] Edge case - empty input
  Input : []  (empty)
  Output: []  (empty)
```

Problem 2: Sort Colors (0, 1, 2)

Question. Given an array containing only 0, 1 and 2, sort it in place. First implement a counting-based solution, then the one-pass Dutch National Flag algorithm without using `std::sort()`.

Approach. Only three key values exist, so comparison sorting is unnecessary. The counting solution tallies the categories and rewrites the array in two passes. The Dutch National Flag solution keeps three regions and classifies each element once, in a single pass.

Algorithm 2 Sort Colors – Counting Based

```
1: function SortColorsCounting(a)
2:     count[0 . . . 2] ← 0
3:     for each value v in a do
4:         count[v] ← count[v] + 1
5:     end for
6:     index ← 0
7:     for c ← 0 to 2 do
8:         for r ← 1 to count[c] do
9:             a[index] ← c
10:            index ← index + 1
11:        end for
12:    end for
13: end function
```

Algorithm 3 Sort Colors – Dutch National Flag (one pass)

```
1: function SortColorsDNF(a)
2:     low ← 0;    mid ← 0;    high ← n − 1
3:     while mid ≤ high do
4:         if a[mid] = 0 then
5:             swap a[low] and a[mid]
6:             low ← low + 1;    mid ← mid + 1
7:         else if a[mid] = 1 then
8:             mid ← mid + 1
9:         else                                          ▷ a[mid] = 2
10:            swap a[mid] and a[high]
11:            high ← high − 1                ▷ mid must not advance
12:        end if
13:    end while
14: end function
```

Why it is correct. Dutch National Flag invariant: $a[0..low-1]$ are 0, $a[low..mid-1]$ are 1, $a[mid..high]$ unexamined, $a[high+1..n-1]$ are 2. On a 2 the value swapped in is unexamined, so mid must not advance. Each iteration decreases $high - mid$, giving at most n iterations.

Complexity. Time: $\Theta(n)$ for both; best, average and worst coincide.
Space: $O(1)$ for both.

Output.

```
[Test 1] Provided example
```

```
  Input          : [2, 0, 2, 1, 1, 0]
  Counting sort  : [0, 0, 1, 1, 2, 2]
  Dutch Nat. Flag: [0, 0, 1, 1, 2, 2]
  Both agree     : yes

[Test 2] Edge case - one category only
  Input          : [2, 2, 2]
  Counting sort  : [2, 2, 2]
  Dutch Nat. Flag: [2, 2, 2]
  Both agree     : yes

[Test 3] Edge case - empty input
  Input          : []  (empty)
  Counting sort  : []  (empty)
  Dutch Nat. Flag: []  (empty)
  Both agree     : yes
```

Problem 3: Largest Number From Concatenation

Question. Given a vector of nonnegative integers, arrange them so that their concatenation forms the largest possible number. Return the answer as a string.

Approach. Numeric order is wrong (3 before 30 gives 330, not 303). Sort on the concatenations instead: a precedes b when $a \frown b > b \frown a$. Both candidates have the same length, so comparing them lexicographically is comparing them numerically.

Algorithm 4 Largest Number From Concatenation

```
1:  function LargestNumber(values)
2:      if values is empty then
3:          return empty string
4:      end if
5:      tokens ← decimal string of each value in values
6:      Sort tokens so that a precedes b whenever a ⌢ b > b ⌢ a
7:      if tokens[0] = "0" then
8:          return "0"                                  ▷ every value is zero
9:      end if
10:     result ← empty string
11:     for each t in tokens do
12:         result ← result ⌢ t
13:     end for
14:     return result
15: end function
```

Why it is correct. The rule is a valid total order because $a \frown b > b \frown a \Leftrightarrow f(a) > f(b)$ with $f(x) = x/(10^{|x|} - 1)$, so it is just sorting on a numeric key. It is optimal by exchange: swapping any adjacent out-of-order pair strictly increases the result. If the largest token is `"0"` every token is zero, so `"0"` is returned rather than `"000"`.

Complexity. Time: $O(nL \log n)$ for n values of at most L digits.
Space: $\Theta(nL)$.

Output.

```
[Test 1] Provided example
  Input : [3, 30, 34, 5, 9]
  Output: "9534330"

[Test 2] Prefix comparison matters
  Input : [10, 2]
  Output: "210"

[Test 3] Edge case - every value is zero
  Input : [0, 0, 0]
  Output: "0"

[Test 4] Edge case - empty input
  Input : []  (empty)
  Output: ""  (empty string)
```

Problem 4: Group Anagrams

Question. Given a vector of strings, group the anagrams together. The order of groups does not matter, but words within each group should retain their original input order.

Approach. Two words are anagrams exactly when their character multisets match, so the word's letters in sorted order form a canonical signature. A hash map from signature to group turns an all-pairs comparison into one pass.

Algorithm 5 Group Anagrams

```
1: function GroupAnagrams(words)
2:     slotOf ← empty map from signature to group index
3:     groups ← empty list of lists
4:     for each word in words, in input order do
5:         signature ← the characters of word in sorted order
6:         if signature is not a key of slotOf then
7:             slotOf[signature] ← number of groups
8:             Append a new group containing only word to groups
9:         else
10:            Append word to groups[ slotOf[signature] ]
11:        end if
12:    end for
13:    return groups
14: end function
```

Why it is correct. Equal sorted forms hold exactly when the multisets are equal, so the signature captures the anagram relation precisely. Words are appended as the input is scanned, which preserves input order inside each group. Duplicates share a signature and are kept as separate entries; empty strings share the signature `""`.

Complexity. Time: $O(nL \log L)$ expected for n words of length at most L.
Space: $\Theta(nL)$.

Output.

```
[Test 1] Provided example
  Input : ["eat", "tea", "tan", "ate", "nat", "bat"]
  Output: [["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]

[Test 2] Duplicates and a repeated word
  Input : ["abc", "cba", "abc", "xyz"]
  Output: [["abc", "cba", "abc"], ["xyz"]]

[Test 3] Edge case - empty string included
  Input : ["", "", "a"]
  Output: [["", ""], ["a"]]

[Test 4] Edge case - empty input
  Input : []  (empty)
  Output: []  (empty)
```

Problem 5: Meeting Rooms – Can One Person Attend Every Meeting?

Question. Given meeting intervals $[start, end]$, determine whether one person can attend every meeting. Meetings that begin exactly when another ends are allowed.

Approach. Sort by start time and compare neighbours only. If $s_i \geq e_{i-1}$ holds for every i then no pair overlaps at all, since $s_j \geq s_{i+1} \geq e_i$ for every $j > i$. The comparison is strict because meetings that touch are allowed.

Algorithm 6 Can One Person Attend Every Meeting?

```
1:  function CanAttendAllMeetings(meetings)
2:      if meetings has fewer than two elements then
3:          return true
4:      end if
5:      Sort meetings by starting value
6:      for i ← 1 to n − 1 do
7:          if meetings[i].start < meetings[i − 1].end then
8:              return false                          ▷ strict: touching is allowed
9:          end if
10:     end for
11:     return true
12: end function
```

Why it is correct. Two meetings conflict when $s_i < e_j \wedge s_j < e_i$, with strict inequalities, so $[1, 5]$ and $[5, 9]$ do not conflict. Sorting by (start, end) also settles the degenerate case: a zero-length meeting sorts before one starting at the same time, which keeps the adjacent scan from reporting a conflict that is not there.

Complexity. Time: $\Theta(n \log n)$ – the sort dominates, so the early exit does not change the bound. Space: $O(\log n)$ auxiliary.

Output.

```
[Test 1] Provided example - overlap exists
  Input : [[0,30], [5,10], [15,20]]
  Output: false

[Test 2] Back-to-back meetings are allowed
  Input : [[1,5], [5,9], [9,12]]
  Output: true

[Test 3] Unsorted input with a hidden conflict
  Input : [[7,10], [2,4], [3,8]]
  Output: false

[Test 4] Edge case - no meetings at all
  Input : []  (empty)
  Output: true
```

Problem 6: K Closest Points to the Origin

Question. Given points (x, y) and an integer k, return the k points closest to the origin. Compare a sorting solution with a heap-based solution.

Approach. Rank by squared distance: the square root is increasing, so the ordering is identical, and the arithmetic stays exact in `long long`. Two solutions are compared – sort all n points and take k, or keep a max-heap of the k best and evict the farthest.

Algorithm 7 K Closest Points – Sorting Solution

```
function KClosestBySorting(points, k)
    if k ≤ 0 or points is empty then
        return empty list
    end if
    k ← min(k, |points|)
    Sort points by x² + y² ascending
    return the first k points
end function
```

Algorithm 8 K Closest Points – Bounded Max-Heap Solution

```
function KClosestByHeap(points, k)
    if k ≤ 0 or points is empty then
        return empty list
    end if
    k ← min(k, |points|)
    heap ← empty max-heap keyed on squared distance
    for each p in points do
        Push (x_p² + y_p², p) onto heap
        if |heap| > k then
            Pop the maximum                    ▷ evict the farthest candidate
        end if
    end for
    result ← all entries of heap, popped and then reversed
    return result                              ▷ nearest first
end function
```

Why it is correct. For the heap, the k closest of the first $i+1$ points are always a subset of the k closest of the first i points together with the new point, so a discarded point never needs to return. Ties may be broken differently by the two methods, so they are compared as multisets of distances rather than as sequences.

Complexity. Time: Sorting $O(n \log n)$; heap $O(n \log k)$. The heap wins when $k \ll n$ and also streams; `nth_element` would give $\Theta(n)$ expected.
Space: Sorting $O(\log n)$; heap $O(k)$.

Output.

```
[Test 1] Provided example
  Input : points = [(1,3), (-2,2)], k = 1
  Sorting solution: [(-2,2)]
  Heap solution   : [(-2,2)]
```

```
  Distances agree : yes

[Test 2] Mixed signs, k in the middle
  Input : points = [(3,3), (5,-1), (-2,4)], k = 2
  Sorting solution: [(3,3), (-2,4)]
  Heap solution   : [(3,3), (-2,4)]
  Distances agree : yes

[Test 3] Edge case - k equals n
  Input : points = [(0,1), (1,0)], k = 2
  Sorting solution: [(0,1), (1,0)]
  Heap solution   : [(1,0), (0,1)]
  Distances agree : yes

[Test 4] Edge case - k is zero
  Input : points = [(1,1), (2,2)], k = 0
  Sorting solution: []  (empty)
  Heap solution   : []  (empty)
  Distances agree : yes
```

Problem 7: Top K Frequent Elements

Question. Given an integer array and an integer k, return the k most frequent values. If frequencies tie, return the smaller value first.

Approach. Count with a hash map, then sort the d distinct (value, count) pairs by count descending and value ascending. Sorting the distinct values rather than all n elements is what keeps this cheap.

Algorithm 9 Top K Frequent Elements

```
function TopKFrequent(values, k)
    if k ≤ 0 or values is empty then
        return empty list
    end if
    frequency ← empty map from value to count
    for each v in values do
        frequency[v] ← frequency[v] + 1
    end for
    items ← the (value, count) pairs of frequency
    k ← min(k, |items|)
    Sort items by count descending; break ties by value ascending
    return the values of the first k entries of items
end function
```

Why it is correct. The comparator orders pairs by $(-\text{count}, \text{value})$. Values are map keys and therefore distinct, so no two entries compare equal, the order is total, and the result is deterministic – the tie rule is honoured both in which values are chosen and in the output order.

Complexity. Time: $O(n + d \log d)$ expected, bounded by $O(n \log n)$ since $d \leq n$.
Space: $\Theta(d)$.

Output.

```
[Test 1] Provided example
  Input : [1, 1, 1, 2, 2, 3], k = 2
  Output: [1, 2]

[Test 2] Frequency tie - smaller value first
  Input : [5, 5, 4, 4, 3], k = 2
  Output: [4, 5]

[Test 3] Edge case - k larger than distinct set
  Input : [9, 9, 9], k = 5
  Output: [9]

[Test 4] Edge case - empty input
  Input : []  (empty), k = 3
  Output: []  (empty)
```

Problem 8: Merge K Sorted Arrays

Question. Given k sorted integer arrays, merge them into one sorted array. Design a solution that does not repeatedly sort the complete output.

Approach. Each array is already sorted, so the smallest unconsumed element is always the head of some array. A min-heap holding one head per array answers that in $O(\log k)$ and produces the output in order, so the complete result is never re-sorted.

Algorithm 10 Merge K Sorted Arrays

```
1:  function MergeKSortedArrays(lists)
2:      heap ← empty min-heap keyed on value
3:      for i ← 0 to k − 1 do
4:          if lists[i] is not empty then
5:              Push ( lists[i][0], i, 0 ) onto heap
6:          end if
7:      end for
8:      merged ← empty list
9:      while heap is not empty do
10:         (value, listIndex, elemIndex) ← pop the minimum of heap
11:         Append value to merged
12:         nextIndex ← elemIndex + 1
13:         if nextIndex < |lists[listIndex]| then
14:             Push ( lists[listIndex][nextIndex], listIndex, nextIndex ) onto heap
15:         end if
16:     end while
17:     return merged
18: end function
```

Why it is correct. Invariant: the heap holds each non-exhausted array's smallest unconsumed element, so the global minimum is always in it and extract-min returns it. Every element is pushed once and popped once, giving exactly N iterations. Empty arrays are never pushed.

Complexity. Time: $O(N \log k)$ against $O(N \log N)$ for concatenate-then-sort.
Space: $O(k)$ auxiliary, plus the $\Theta(N)$ output.

Output.

```
[Test 1] Provided example
  Input : [[1, 4, 5], [1, 3, 4], [2, 6]]
  Output: [1, 1, 2, 3, 4, 4, 5, 6]

[Test 2] Very uneven lengths with duplicates
  Input : [[-5], [0, 0, 0], [-9, -4, 7, 7]]
  Output: [-9, -5, -4, 0, 0, 0, 7, 7]

[Test 3] Edge case - some lists are empty
  Input : [[], [2, 8], []]
  Output: [2, 8]

[Test 4] Edge case - every list is empty
  Input : [[], []]
  Output: []  (empty)
```

Problem 9: Kth Largest Element

Question. Given an unsorted integer array and k, return the kth-largest element. Duplicate values count as separate positions.

Approach. The kth largest sits at ascending index $n - k$, so only one order statistic is needed. Quickselect partitions and recurses into one side only. A three-way partition groups all copies of the pivot at once, which keeps duplicate-heavy input linear, and a random pivot means no input can force the worst case.

Algorithm 11 Three-Way Partition (Dutch National Flag)

```
1:  function ThreeWayPartition(a, low, high)
2:      pivot ← a[ random index in [low, high] ]
3:      lt ← low;    gt ← high;    i ← low
4:      while i ≤ gt do
5:          if a[i] < pivot then
6:              swap a[i] and a[lt];    i ← i + 1;    lt ← lt + 1
7:          else if a[i] > pivot then
8:              swap a[i] and a[gt];    gt ← gt − 1
9:          else
10:             i ← i + 1
11:         end if
12:     end while
13:     return (lt, gt)                          ▷ a[lt ... gt] all equal pivot
14: end function
```

Algorithm 12 Kth Largest Element (Quickselect)

```
1:  function KthLargest(values, k)
2:      if k < 1 or k > n then
3:          report an error
4:      end if
5:      target ← n − k                           ▷ index in ascending order
6:      low ← 0;    high ← n − 1
7:      while low < high do
8:          (lt, gt) ← ThreeWayPartition(values, low, high)
9:          if target < lt then
10:             high ← lt − 1
11:         else if target > gt then
12:             low ← gt + 1
13:         else
14:             return values[target]             ▷ inside the pivot block
15:         end if
16:     end while
17:     return values[low]
18: end function
```

Why it is correct. After partitioning, $a[lt..gt]$ all equal the pivot and are in final position, so the target is either strictly left, strictly right, or already settled. The pivot block is never empty, so the range strictly shrinks each iteration. k outside $[1, n]$ is rejected.

Complexity. Time: $\Theta(n)$ best and expected; $\Theta(n^2)$ worst, unreachable by choice of input. Space: $O(1)$ auxiliary – the search loop is iterative.

Output.

```
[Test 1] Provided example
  Input : [3, 2, 1, 5, 6, 4], k = 2
  Output: 5

[Test 2] Duplicates count as separate positions
  Input : [3, 2, 3, 1, 2, 4, 5, 5, 6], k = 4
  Output: 4

[Test 3] Edge case - k equals n (the minimum)
  Input : [7, 7, 2], k = 3
  Output: 2

[Test 4] Edge case - single element
  Input : [42], k = 1
  Output: 42

[Test 5] Edge case - k out of range
  Input : [1, 2, 3], k = 7
  Output: rejected (k must satisfy 1 <= k <= n)
```

Problem 10: Count Inversions

Question. Count pairs (i, j) for which $i < j$ and $values[i] > values[j]$. Develop an efficient solution by modifying Merge Sort, in $O(n \log n)$ time, using a 64-bit integer for the count.

Approach. Every inversion lies wholly in the left half, wholly in the right half, or crosses between them, and these three classes are disjoint. Merge Sort counts the crossing ones for free: when a right element is taken while $a[i..mid]$ are still pending, all $mid - i + 1$ of them exceed it, so a whole batch is counted with one addition.

Algorithm 13 Merge Two Sorted Halves and Count Cross Inversions

```
1: function MergeAndCount(a, buffer, left, mid, right)
2:     count ← 0
3:     i ← left;    j ← mid + 1;    t ← left
4:     while i ≤ mid and j ≤ right do
5:         if a[i] ≤ a[j] then
6:             buffer[t] ← a[i];    i ← i + 1                ▷ ≤ keeps equal pairs uncounted
7:         else
8:             count ← count + (mid − i + 1)                 ▷ a[i ... mid] all exceed a[j]
9:             buffer[t] ← a[j];    j ← j + 1
10:        end if
11:        t ← t + 1
12:    end while
13:    Copy any remaining elements of both halves into buffer
14:    Copy buffer[left ... right] back into a[left ... right]
15:    return count
16: end function
```

Algorithm 14 Count Inversions by Modified Merge Sort

```
1: function SortAndCount(a, buffer, left, right)
2:     if left ≥ right then
3:         return 0                                          ▷ zero or one element has no inversion
4:     end if
5:     mid ← left + ⌊(right − left)/2⌋
6:     count ← SortAndCount(a, buffer, left, mid)
7:     count ← count+ SortAndCount(a, buffer, mid + 1, right)
8:     count ← count+ MergeAndCount(a, buffer, left, mid, right)
9:     return count
10: end function

11: function CountInversions(values)
12:     if |values| < 2 then
13:         return 0
14:     end if
15:     buffer ← array of size |values|
16:     return SortAndCount(values, buffer, 0, |values| − 1)
17: end function
```

Why it is correct. Merge Sort permutes only within a half, so every left element keeps an original

index below every right element and the counted pairs are genuine. Taking from the left on ties ($a[i] \leq a[j]$) means equal values are not counted, matching the strict definition. The count is `long long` because it reaches $n(n-1)/2$, which overflows 32 bits near $n = 65{,}536$.

Complexity. Time: $\Theta(n \log n)$ from $T(n) = 2T(n/2) + \Theta(n)$; independent of input order. Brute force is $\Theta(n^2)$ and is used only to validate.
Space: $\Theta(n)$ for one reused buffer, plus $\Theta(\log n)$ stack.

Output.

```
[Test 1] Provided example
  Input          : [2, 4, 1, 3, 5]
  Output         : 3
  Brute force    : 3  -> match

[Test 2] Reverse-sorted input (worst case)
  Input          : [5, 4, 3, 2, 1]
  Output         : 10
  Brute force    : 10  -> match

[Test 3] Duplicates are not inversions
  Input          : [2, 2, 2, 2]
  Output         : 0
  Brute force    : 0  -> match

[Test 4] Edge case - empty input
  Input          : []  (empty)
  Output         : 0
  Brute force    : 0  -> match
```